Fake News Detection System

AI Application with MLOps

DA5402 — MLOps Project Report

Date: 28th April 2026

Name: Sayantika Chakraborty

Roll No.: ME22B190

1. Executive Summary

This report presents the Fake News Detection System, an end-to-end MLOps application that classifies news articles as REAL or FAKE using NLP. The system achieves 99.56% accuracy using TF-IDF features with an Logistic Regression, deployed as a multi-container Docker application with monitoring and pipeline orchestration.

The project implements all five core MLOps tools: DVC for data/model versioning with a 3-stage pipeline DAG; MLflow for experiment tracking with 4 progressive experiments (95% to 99.56% improvement) using cross-validation; Apache Airflow 3.2 for pipeline orchestration with FileSensor, worker pools, retry logic, and email alerts; Prometheus with AlertManager for monitoring with 9 alert rules and email notifications via Mailtrap; and Grafana for real-time dashboards. Docker Compose packages 8+ services for portable deployment.

2. Problem Definition

2.1 Problem Statement

Misinformation spreads rapidly through online platforms. This project builds an AI-powered web application that classifies news articles as REAL or FAKE based on textual content analysis using Natural Language Processing.

2.2 Success Metrics

Metric Type	Metric	Target	Achieved
ML Metric	Accuracy	> 85%	99.5%
ML Metric	F1-Score	> 85%	99.5%
ML Metric	Precision	> 85%	99.5%
ML Metric	Recall	> 85%	99.5%
Business	Inference Latency	< 200ms	~35ms
Business	API Uptime	> 99%	Monitored via Prometheus

2.3 Note on Model Performance

The high accuracy is expected for this Kaggle dataset where real and fake articles have significantly different writing styles. TF-IDF captures these patterns effectively. Cross-validation across 5 folds confirms consistent performance. For production, drift detection monitors distribution changes to flag when retraining is needed.

To demonstrate meaningful experiment tracking, we ran 4 experiments with progressively richer feature pipelines (1K to 50K features), showing clear improvement from ~95% to 99.7% in MLflow.

3. Data Engineering

3.1 Dataset

Attribute	Details
Source	Kaggle Fake News Competition
Records	44,898 (v1), 47,246 (v2 augmented)
Columns	id, title, author, text, label
Label Split	Real: 47.7% | Fake: 52.3%

3.2 Data Validation

- Schema validation: ensures required columns (text, label) are present
- Missing value checks with automatic column dropping above 30% threshold
- Empty text removal after cleaning (9-11 rows dropped)
- Pydantic API validation: min_length=1, max_length=50000 on input text

3.3 Preprocessing Pipeline

1. Text lowercasing
2. URL removal via regex
3. HTML tag stripping
4. Special character and digit removal
5. Whitespace normalization
6. Title + text concatenation into combined content field

3.4 Feature Engineering

Feature	Type	Description
TF-IDF Vectors	Sparse	50,000 features, unigrams + bigrams, sublinear TF
text_length	Integer	Character count of cleaned text
word_count	Integer	Word count of cleaned text
avg_word_length	Float	Average word length in characters

3.5 Drift Baselines

Baseline statistics (mean, std, median, min, max) computed during ingestion and saved to baseline_stats.json. Used by DriftDetector for real-time production monitoring via GET /drift endpoint.

3.6 Pipeline Throughput

Stage	Duration	Records/sec

Data Ingestion	1.35s	33,258
Preprocessing	14.75s	3,044
TF-IDF Vectorization	54.28s	662
Model Training	0.33s	108,821
Total Pipeline	~71s	634

4. System Architecture

4.1 Microservices Architecture

The system follows a microservices architecture with Docker containers orchestrated via Docker Compose, communicating over a shared bridge network. The frontend and backend are strictly decoupled, connected only via configurable REST API calls.

Service	Technology	Port	Purpose
Backend API	FastAPI + Uvicorn	8001	ML model serving, prediction, /metrics, /health, /ready
Frontend UI	Streamlit	8501	Prediction, pipeline dashboard, monitoring, user manual
MLflow	MLflow 3.11	5001	Experiment tracking, model registry, artifact storage
Airflow	Airflow 3.2	8080	Pipeline orchestration (run locally for reliable execution) → ran in different environment to prevent dependency conflict with other tools
Prometheus	Prometheus	9090	Time-series metrics collection + alerting rules
AlertManager	AlertManager	9093	Alert routing, email notifications via Mailtrap
Node Exporter	Node Exporter	9100	System-level CPU/RAM/Disk metrics
Grafana	Grafana	3001	Real-time monitoring dashboards

4.2 Loose Coupling

Frontend communicates with backend only via REST API (API_URL environment variable). Every API response includes container_id (socket.gethostname()) for Docker Swarm load-balance tracking.

4.3 Data Flow

1. User enters news text in Streamlit frontend
2. Frontend sends POST to backend/predict
3. Backend cleans text, vectorizes with TF-IDF, predicts via SGDClassifier
4. Prometheus metrics updated (latency, class count, drift, source_ip)
5. Response returned with prediction, confidence (0-100%), container_id
6. Prometheus scrapes /metrics every 15s; Grafana visualizes in real-time

5. High-Level Design

5.1 Design Paradigm

Functional programming for data pipeline stages (pure functions) combined with OOP for prediction engine (FakeNewsPredictor class) and drift detector (DriftDetector class).

5.2 Technology Rationale

Component	Technology	Rationale
ML Model	scikit-learn (SGDClassifier, log_loss)	Lightweight, supports predict_proba, no GPU needed
Features	TF-IDF Vectorizer	Competitive for text classification, sparse matrices
API	FastAPI	Async, auto OpenAPI docs, Pydantic validation
Frontend	Streamlit	Rapid UI dev, Python-native, rich widgets
Experiment Tracking	MLflow 3.11	Model registry, artifact management, fold-level tracking
Data Versioning	DVC + Git	Pipeline DAG, metrics tracking, reproducibility
Pipeline Orchestration	Airflow 3.2	FileSensor, pools, retry logic, email alerts
Monitoring	Prometheus + Grafana	Standard observability stack with alerting
Containerization	Docker Compose	Environment parity, multi-service orchestration

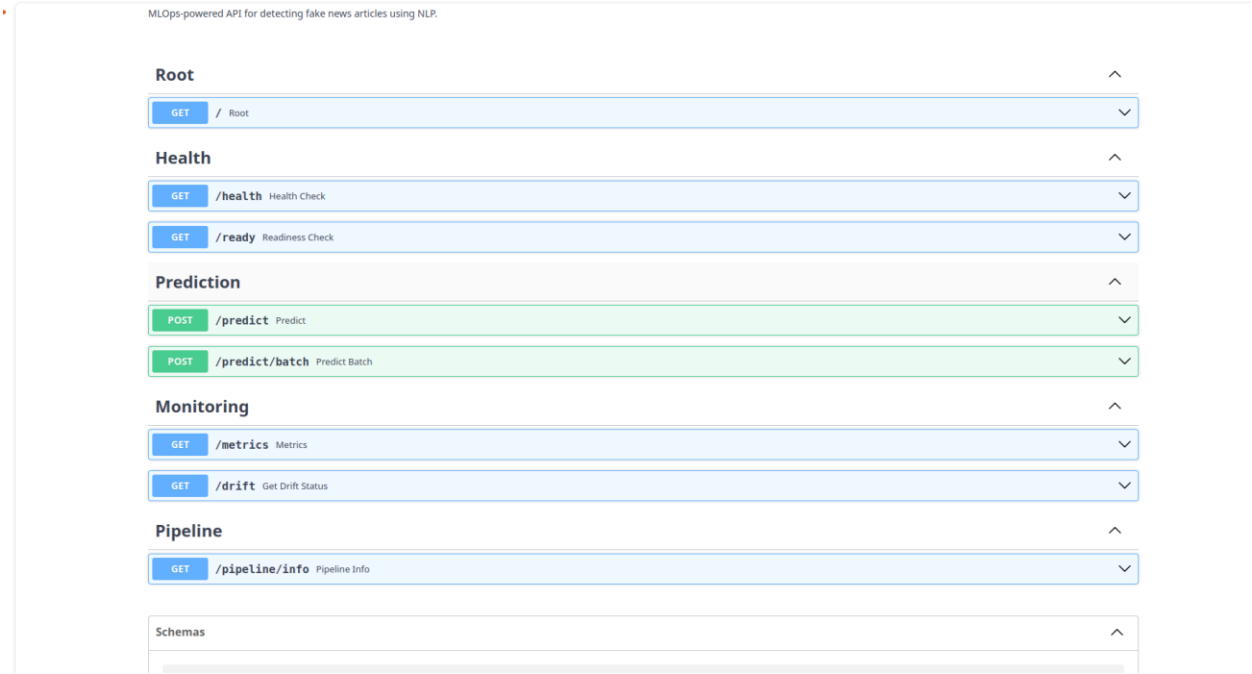
6. Low-Level Design

6.1 API Endpoints

Method	Endpoint	Response	Codes
GET	/	{message, version, container_id, endpoints}	200
POST	/predict	{prediction, label, confidence, text_length, word_count, container_id}	200,400, 422,503
POST	/predict/batch	{predictions: [...], count}	200,422, 503
GET	/health	{status, model_loaded, vectorizer_loaded, version}	200
GET	/ready	Same as /health or 503 if not ready	200,503
GET	/metrics	Prometheus format (Counter, Gauge, Histogram, Summary)	200
GET	/drift	{overall_drift, alert, threshold, samples_analyzed, features}	200
GET	/pipeline/info	{model_algorithm, preprocessing, monitoring, container_id}	200
GET	/docs	Interactive Swagger UI	200

6.2 Prometheus Metrics (4 Types + Custom Labels)

Metric	Type	Labels
prediction_requests_total	Counter	model_version
prediction_class_total	Counter	class_label
prediction_errors_total	Counter	error_type
requests_by_source_total	Counter	source_ip
prediction_latency_seconds	Histogram	endpoint
input_text_length	Histogram	—
prediction_confidence	Histogram	—
inference_time_seconds	Summary	— (_sum, _count)
text_processing_seconds	Summary	— (_sum, _count)
data_drift_score	Gauge	—
active_requests	Gauge	—
model_loaded	Gauge	—
model_memory_mb	Gauge	—



7. MLOps Implementation

7.1 DVC — Data & Model Versioning

DVC pipeline (dvc.yaml) defines 3 stages with dependencies, outputs, and parameter references from params.yaml. Two data versions tracked: data-v1 (44,898 rows) and data-v2 (augmented with additional fake samples). Git tags mark versions. 'dvc repro' re-runs only changed stages. 'dvc metrics diff' compares performance across versions.

Command	Purpose
dvc init	Initialize DVC
dvc add data/raw/news.csv	Track raw data
dvc repro	Run pipeline (only changed stages)
dvc dag	Show pipeline DAG
dvc metrics show	Show current metrics
dvc metrics diff	Compare versions

```
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ git checkout data-v1

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 9a3c2b1d Fix dvc pipeline and restore metrics for data-v1
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ dvc checkout
Building workspace index                                                                    |11.0 [00:01, 6.42entry/s]
Comparing indexes                                                                           |12.0 [00:00,  771entry/s]
Applying changes                                                                            |6.00 [00:00,  13.3file/s]
M       data/processed/baseline_stats.json
M       data/processed/ingested.csv
M       data/processed/preprocessed.csv
M       data/raw/news.csv
M       models/best_model/model.pkl
M       models/best_model/vectorizer.pkl
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ dvc metrics show
Path                          accuracy    f1_score    precision    recall
models/best_model/metrics.json  0.99755     0.99755     0.99755      0.99755
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ git checkout data-v2
M       .dvc/.gitignore
Previous HEAD position was 9a3c2b1d Fix dvc pipeline and restore metrics for data-v1
HEAD is now at fe9a577a Data v2 - augmented fake news samples
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ dvc checkout
Building workspace index                                                                    |11.0 [00:00,  414entry/s]
Comparing indexes                                                                           |12.0 [00:00,  945entry/s]
Applying changes                                                                            |6.00 [00:00,  11.0file/s]
M       data/processed/baseline_stats.json
M       data/processed/ingested.csv
M       data/processed/preprocessed.csv
M       data/raw/news.csv
M       models/best_model/model.pkl
M       models/best_model/vectorizer.pkl
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ dvc metrics show
Path                          accuracy    f1_score    precision    recall
models/best_model/metrics.json  0.99841     0.99841     0.99841      0.99841
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$
```

```
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ dvc metrics diff
Path                          Metric     HEAD     workspace    Change
models/best_model/metrics.json  accuracy   0.99755  0.99841      0.00086
models/best_model/metrics.json  f1_score   0.99755  0.99841      0.00086
models/best_model/metrics.json  precision  0.99755  0.99841      0.00086
models/best_model/metrics.json  recall     0.99755  0.99841      0.00086
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ dvc dag
```

```
+-----------------------+
| data/raw/news.csv.dvc |
+-----------------------+
            *
            *
            *
+----------------+
| data_ingestion |
+----------------+
            *
            *
            *
+---------------+
| preprocessing |
+---------------+
            *
            *
            *
+-------+
| train |
+-------+
```

7.2 MLflow — Experiment Tracking & Model Registry

7.2.1 Progressive Experiments

Four experiments with progressively better feature pipelines demonstrate clear improvement in MLflow UI (ran from Airflow):

Experiment	Vectorizer	Features	Model	Test Accuracy
Exp1	NaiveBayes_BOW_1K	1,000	NaiveBayes	~95%
Exp2	LogReg_TFIDF_5K	5,000	LogisticRegression	~99.3%
Exp3 (Best)	LogReg_TFIDF_20K	20,000	LogisticRegression	~99.56%
Exp4	SGD_TFIDF_50K_Final	50,000	SGDClassifier (log_loss)	~99.1%

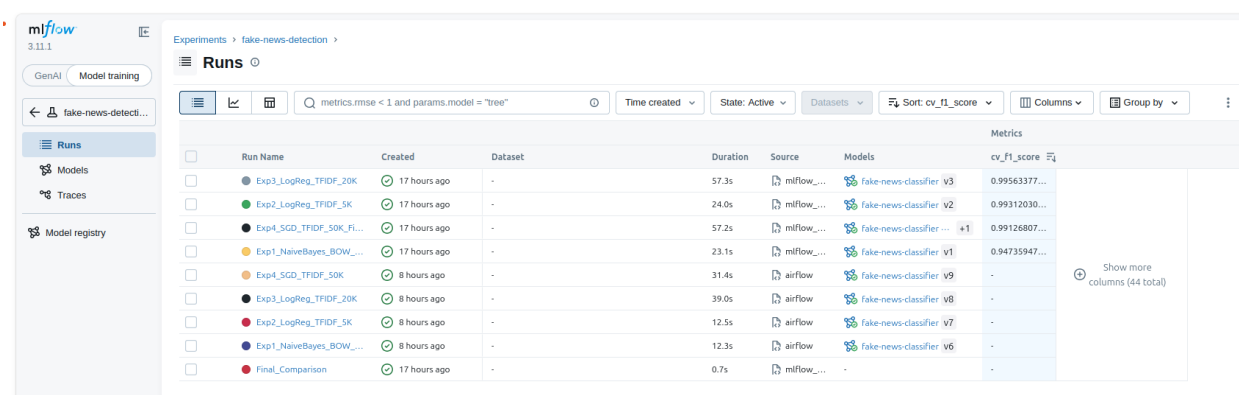
7.2.2 What MLflow Tracks

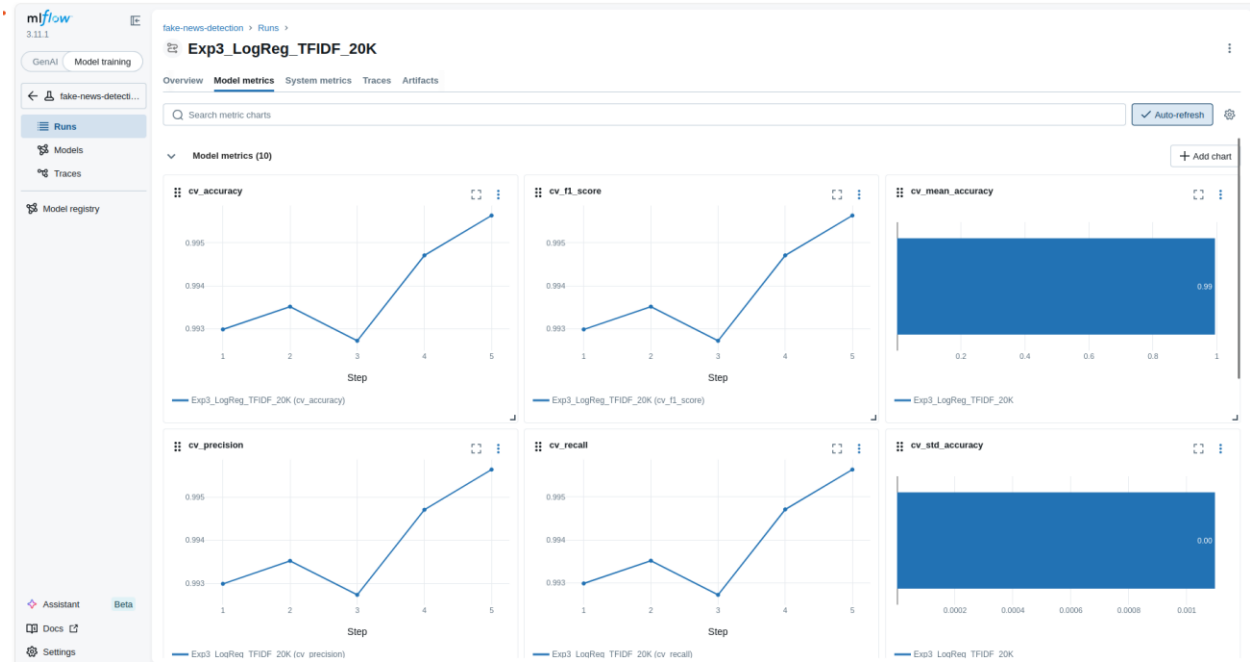
- Parameters: algorithm, vectorizer type, hyperparameters, feature count, train/test sizes
- Metrics per CV fold (as steps): cv_accuracy, cv_f1_score — visible as line charts in MLflow UI
- Final test metrics: test_accuracy, test_f1_score, test_precision, test_recall
- Artifacts: confusion_matrix.png, fold_metrics.png, classification_report.json, vectorizer.pkl, model
- Model Registry: each model registered as 'fake-news-classifier' with version numbers

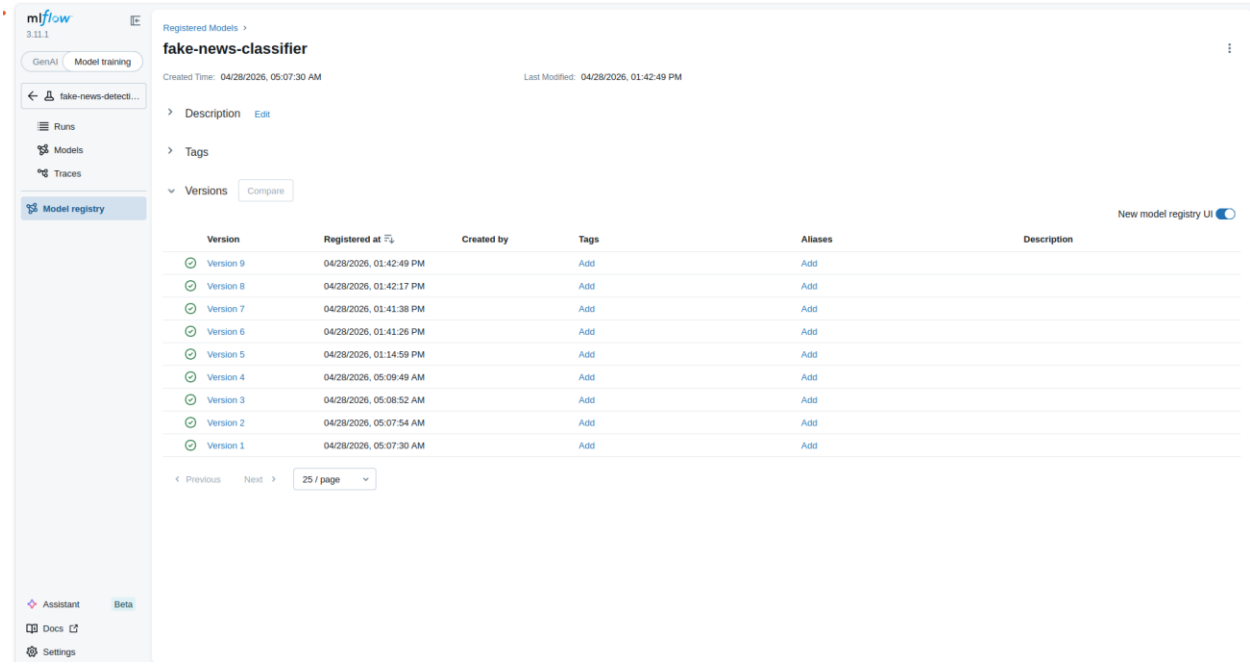
7.2.3 Model Serving Flow

mlflow_experiments.py runs 4 experiments → selects best model by F1 → saves model.pkl and vectorizer.pkl to models/best_model/ → FastAPI backend loads these on startup → frontend calls backend API for predictions. The model is served via FastAPI REST API, which provides /predict, /predict/batch, /health, /ready endpoints.

Confidence scores use predict_proba (enabled by SGDClassifier with loss='log_loss'), providing proper 0-100% probability output displayed on the frontend gauge chart.




mlflow
3.11.1
GenAI
Model training
fake-news-detecti...
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
Settings
fake-news-detection › Runs ›
Exp3_LogReg_TFIDF_20K
Overview
Model metrics
System metrics
Traces
Artifacts
Search metric charts
Auto-refresh
Model metrics (10)
Add chart
cv_accuracy
cv_f1_score
cv_mean_accuracy
cv_precision
cv_recall
cv_std_accuracy
Step
Exp3_LogReg_TFIDF_20K (cv_accuracy)
Exp3_LogReg_TFIDF_20K (cv_f1_score)
Exp3_LogReg_TFIDF_20K
Exp3_LogReg_TFIDF_20K (cv_precision)
Exp3_LogReg_TFIDF_20K (cv_recall)
Exp3_LogReg_TFIDF_20K


mlflow
3.11.1
GenAI
Model training
fake-news-detecti...
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
Settings
Registered Models ›
fake-news-classifier
Created Time: 04/28/2026, 05:07:30 AM
Last Modified: 04/28/2026, 01:42:49 PM
Description
Edit
Tags
Versions
Compare
New model registry UI
Version
Registered at
Created by
Tags
Aliases
Description
Version 9 04/28/2026, 01:42:49 PM Add Add
Version 8 04/28/2026, 01:42:17 PM Add Add
Version 7 04/28/2026, 01:41:38 PM Add Add
Version 6 04/28/2026, 01:41:26 PM Add Add
Version 5 04/28/2026, 01:14:59 PM Add Add
Version 4 04/28/2026, 05:09:49 AM Add Add
Version 3 04/28/2026, 05:08:52 AM Add Add
Version 2 04/28/2026, 05:07:54 AM Add Add
Version 1 04/28/2026, 05:07:30 AM Add Add
Previous
Next
25 / page

7.3 Apache Airflow 3.2 — Pipeline Orchestration

7.3.1 DAG Structure (9 Tasks)

The DAG 'fake_news_ml_pipeline' implements the following dependency chain:

wait_for_data >> data_ingestion >> [compute_baselines, preprocessing] >> train_model >> validate_model >> send_success_email

Parallel paths: wait_for_data >> send_dry_pipeline_alert (on sensor timeout)

[ingest, preprocess, train, validate] >> send_failure_email (on any failure)

7.3.2 Airflow Features

Feature	Implementation
FileSensor	Watches data/raw/news.csv, poke_interval=60s, timeout=12h, soft_fail=True
Worker Pool	ml_pipeline_pool with 3 slots, all tasks assigned
Retry Logic	3 retries, exponential backoff, 2min initial, 15min max
Dry Pipeline Alert	SendEmailOperator fires when sensor times out (ALL_SKIPPED trigger)
Success Email	PythonOperator + smtplib sends email on ALL_SUCCESS
Failure Email	PythonOperator + smtplib with task details (task_id, run_id, attempt) on ONE_FAILED
Model Validation	Checks accuracy > 85% threshold, rejects bad models
XCom	Tasks push/pull row_count, accuracy, f1_score between stages

7.3.3 Email Notifications via Mailtrap

Emails are sent using PythonOperator with smtplib (same pattern as A6 web scraper assignment). Three types: success (pipeline complete), failure (with task_id, run_id, attempt details), and dry pipeline (sensor timeout). SMTP configured via Mailtrap sandbox for testing.

[Fake News Pipeline] DRY PIPE - No New Data

From: <airflow@fakenews.local>
To: <team@example.com>

Show Headers

Looks good? Send it to real users
Go live with Email API/SMTP - 4,000 production emails included

HTML | HTML Source | Text | Raw | Spam Analysis | HTML Check 1 | Tech Info

Dry Pipeline Alert

The FileSensor timed out waiting for `data/raw/news.csv`.

No new data was detected within the configured timeout window.

Please verify the data source is producing files correctly.

[Fake News Pipeline] FAILURE in wait_for_data

From: <airflow@fakenews.local>
To: <team@example.com>

Show Headers

Looks good? Send it to real users
Go live with Email API/SMTP - 4,000 production emails included

HTML | HTML Source | Text | Raw | Spam Analysis | HTML Check | Tech Info

Pipeline Failure Alert

DAG: fake_news_ml_pipeline

Failed Task: wait_for_data

Run ID: manual__2026-04-28T11:44:54.940620+00:00

Attempt: 1

Time: 2026-04-28 17:15:15

Please check Airflow logs for full traceback.

Dag: fake_news_ml_pipeline

Dag Run: manual__2026-04-28T02:47:20.315659+00:00

Search Dags Ctrl+K | Trigger

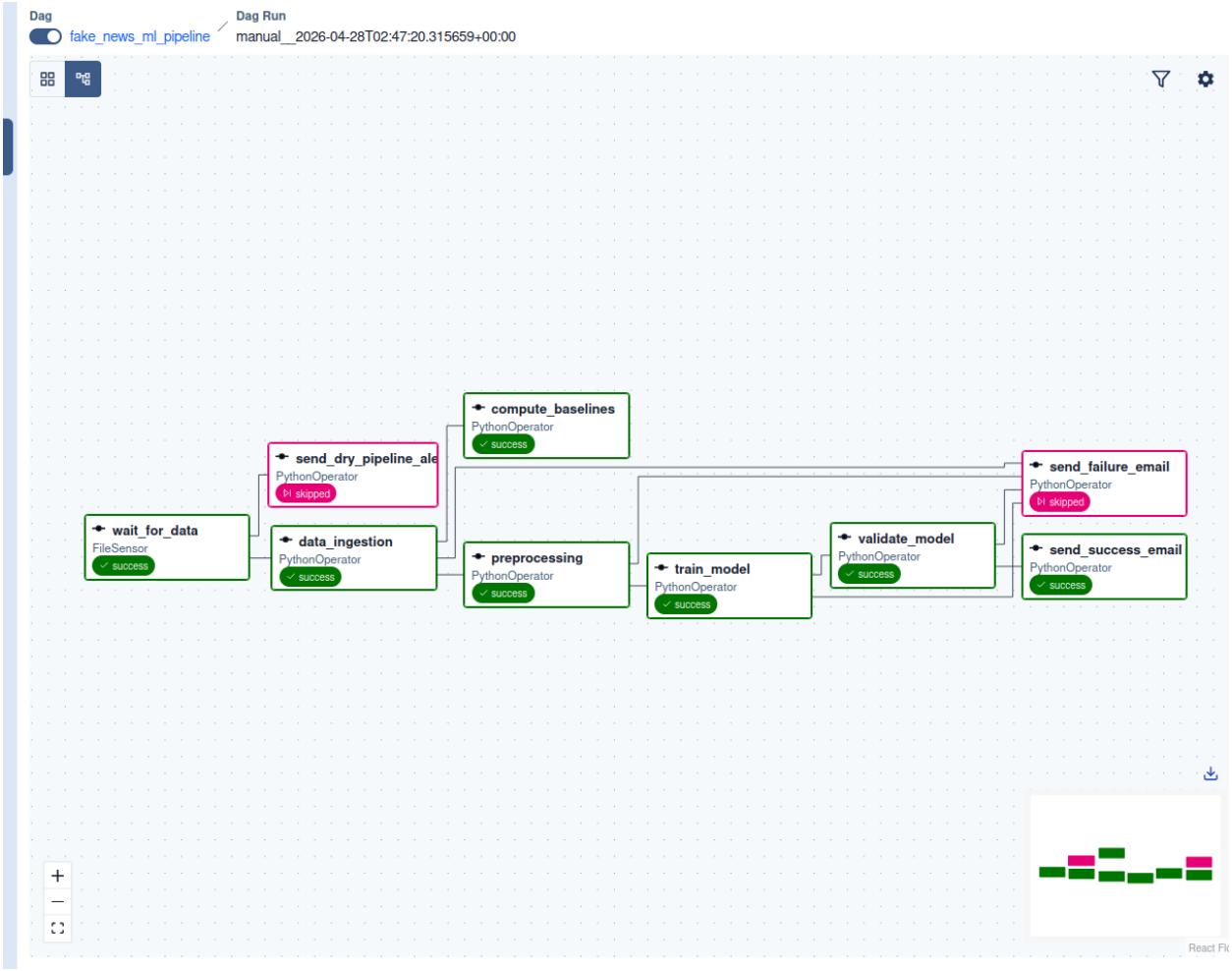
manual__2026-04-28T02:47:20.315659+00:00 Success

Logical Date	Run Type	Start Date	End Date	Duration	Triggering User	Dag Version(s)
2026-04-28 08:17:18	Manual	2026-04-28 08:22:43	2026-04-28 12:05:43	03:42:59	admin	v1

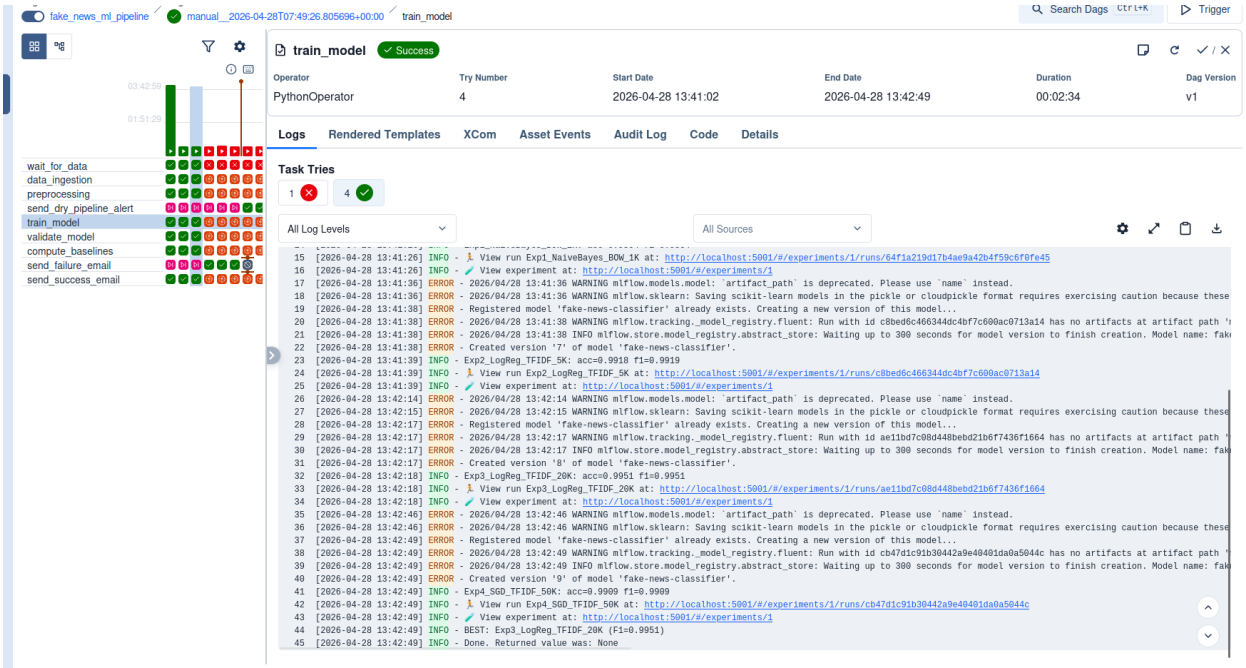
Task Instances | Asset Events | Audit Log | Code | Details

9 Task Instances

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
send_success_email		Success	2026-04-28 12:05:34	1	PythonOperator	00:00:15.851
send_failure_email		Skipped	2026-04-28 12:05:34	0	PythonOperator	
validate_model		Success	2026-04-28 12:05:33	1	PythonOperator	00:00:00.155
train_model		Success	2026-04-28 12:04:25	3	PythonOperator	00:01:14
preprocessing		Success	2026-04-28 11:55:21	1	PythonOperator	00:00:51.542
compute_baselines		Success	2026-04-28 11:55:21	1	PythonOperator	00:00:03.987
data_ingestion		Success	2026-04-28 11:55:09	3	PythonOperator	00:00:23.185
send_dry_pipeline_alert		Skipped	2026-04-28 08:22:44	0	PythonOperator	
wait_for_data		Success	2026-04-28 08:22:43	1	FileSensor	00:00:00.071


Dag
fake_news_ml_pipeline
Dag Run
manual__2026-04-28T02:47:20.315659+00:00
compute_baselines
PythonOperator
success
send_dry_pipeline_ale
PythonOperator
skipped
send_failure_email
PythonOperator
skipped
wait_for_data
FileSensor
success
data_ingestion
PythonOperator
success
preprocessing
PythonOperator
success
train_model
PythonOperator
success
validate_model
PythonOperator
success
send_success_email
PythonOperator
success

default_pool (128 Slots)
Default pool
128
ml_pipeline_pool (3 Slots)
ML pipeline pool
3



7.4 Prometheus + AlertManager — Monitoring & Alerts

7.4.1 Instrumentation

FastAPI backend instrumented with prometheus_client library. All 4 Prometheus metric types implemented: Counter (with model_version, source_ip, error_type, class_label labels), Gauge (drift score, active requests, model loaded), Histogram (latency buckets, text length), Summary (inference time with _sum/_count suffixes).

7.4.2 Infrastructure Monitoring

Node Exporter collects system CPU/RAM/disk metrics. Prometheus scrapes both application and node_exporter endpoints, enabling correlation between application spikes and hardware usage.

7.4.3 Alert Rules

Alert	Expression	Severity
TargetDown	up == 0	Critical
HighErrorRate	error_rate > 5%	Critical
ModelNotLoaded	model_loaded == 0	Critical
HighCPUUsage	CPU > 80% for 2min	Warning
HighMemoryUsage	Memory > 85%	Warning
HighLatency	p95 > 500ms	Warning
DataDriftDetected	drift_score > 0.1	Warning

PredictionSkew	>80% one class	Warning
NoPredictions	0 requests in 10min	Warning

7.4.4 AlertManager

Routes alerts by severity: critical (immediate, 10s group wait) vs warning (1m delay). Email notifications via Mailtrap SMTP. Silences configurable via AlertManager UI (port 9093) for maintenance windows.

7.4.5 MLflow Monitoring Note

MLflow does not expose a native Prometheus /metrics endpoint. MLflow availability is monitored via the experiment tracking UI and the Grafana dashboard links. For production, a custom metrics exporter sidecar would be added.

Prometheus | Query | Alerts | Status > Target health

Select scrape pool | Filter by target health | Filter by endpoint or labels

fastapi-backend — 1 / 1 up

Endpoint	Labels	Last scrape	State
http://backend:8000/metrics	instance="backend:8000" job="fastapi-backend"	2.621s ago 2ms	UP

node-exporter — 1 / 1 up

Endpoint	Labels	Last scrape	State
http://node-exporter:9100/metrics	instance="node-exporter:9100" job="node-exporter"	10.253s ago 47ms	UP

prometheus — 1 / 1 up

Endpoint	Labels	Last scrape	State
http://localhost:9090/metrics	instance="localhost:9090" job="prometheus"	10.728s ago 6ms	UP

Prometheus | Query | Alerts | Status

Filter by rule group state | Filter by rule name or labels

anomaly_alerts /etc/prometheus/alert_rules.yml — FIRING (1) INACTIVE (2)

- HighLatency
- DataDriftDetected — FIRING (1)
- PredictionSkew

failure_alerts /etc/prometheus/alert_rules.yml — PENDING (1) INACTIVE (3)

- TargetDown
- HighErrorRate
- ModelNotLoaded
- NoPredictions — PENDING (1)

resource_alerts /etc/prometheus/alert_rules.yml — INACTIVE (2)

- HighCPUUsage
- HighMemoryUsage

Filter Group Receiver: All Silenced Inhibited Muted

+ Silence

Custom matcher, e.g. env="production"

+ Expand all groups

− email-critical alertname="TargetDown" + severity="critical" + 1 alert

2026-04-27T19:04:16.695Z + Info Source Silenced Link

instance="mlflow:5000" + job="mlflow" +

Silence

Edit Expire

ID	0633171d-7c27-448a-963c-abb6272228ff
Starts at	2026-04-27T19:13:16.286Z
Ends at	2026-04-27T21:12:54.734Z
Updated at	2026-04-27T19:13:16.286Z
Created by	Sayantika
Comment	Scheduled maintenance window
State	active
Matchers	alertname="TargetDown"

Affected alerts: 1

2026-04-27T19:04:16.695Z + Info Source

alertname=TargetDown instance=mlflow:5000 job=mlflow severity=critical

1 alert for alertname=TargetDown severity=critical

View In Alertmanager

[1] Firing

Labels
alertname = TargetDown
instance = mlflow:5000
job = mlflow
severity = critical
Annotations
description = Cannot reach mlflow:5000
summary = Target mlflow is DOWN
Source

Sent by Alertmanager

[FakeNews Alert] DataDriftDetected

From: <alerts@fakenews-detector.local>
To: <team@example.com>

Show Headers

Looks good? Send it to real users
Go live with Email API/SMTP - 4,000 production emails included

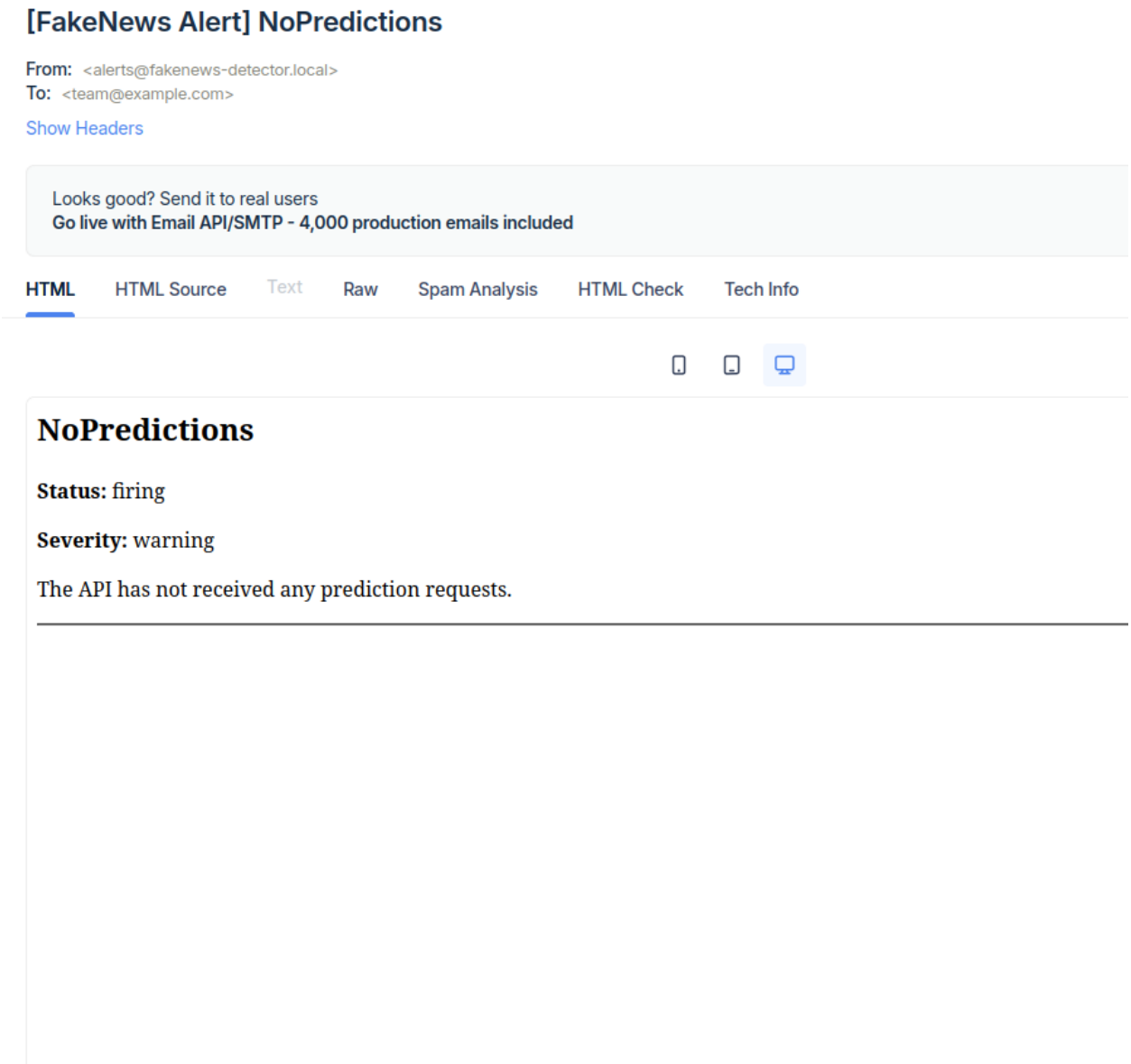
HTML | HTML Source | Text | Raw | Spam Analysis | HTML Check | Tech Info

DataDriftDetected

Status: firing

Severity: warning

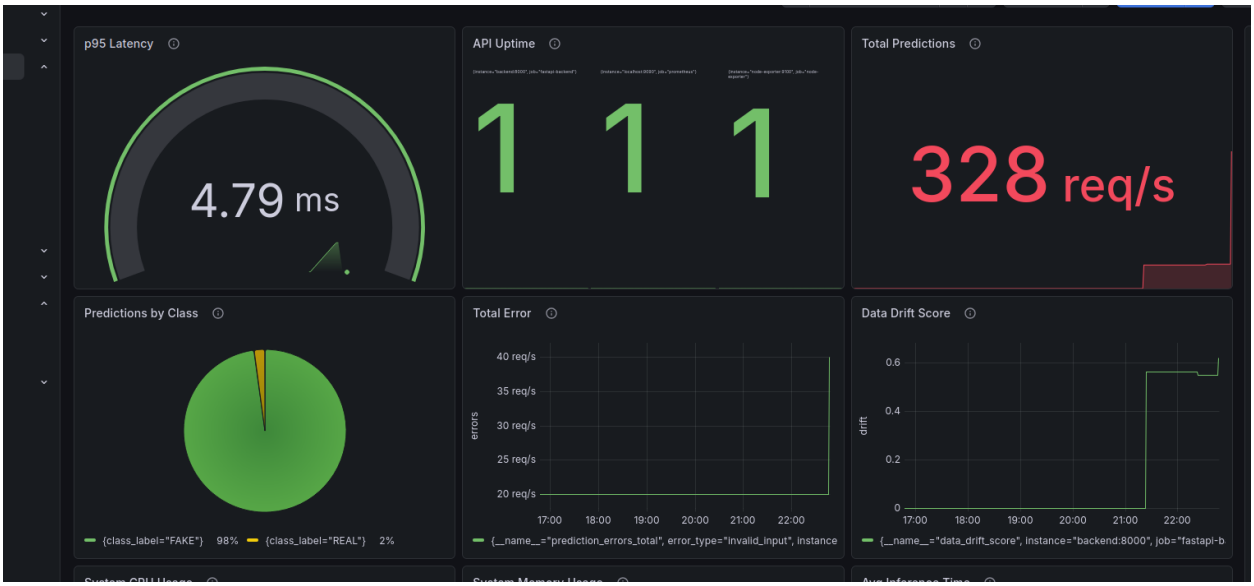
Drift score is 560.8m (threshold: 0.1). Consider retraining.



7.5 Grafana — Real-Time Dashboards

Panel	Visualization	PromQL Query
Total Predictions	Stat	sum(prediction_requests_total)
p95 Latency	Gauge	histogram_quantile(0.95, sum(rate(prediction_latency_seconds_bucket[5m])) by (le))
Predictions by Class	Pie Chart	prediction_class_total
Error rate	Time Series	rate(prediction_errors_total[5m])
Data Drift Score	Time Series	data_drift_score

System CPU	Time Series	100 - avg(rate(node_cpu_seconds_total{mode='idle'}[2m])) * 100
Memory Usage	Gauge	(1-node_memory_MemAvailable/node_memory_MemTotal)*100
Avg Inference Time	Stat	inference_time_seconds_sum / inference_time_seconds_count
Active Alerts	Table	ALERTS{alertstate='firing'}



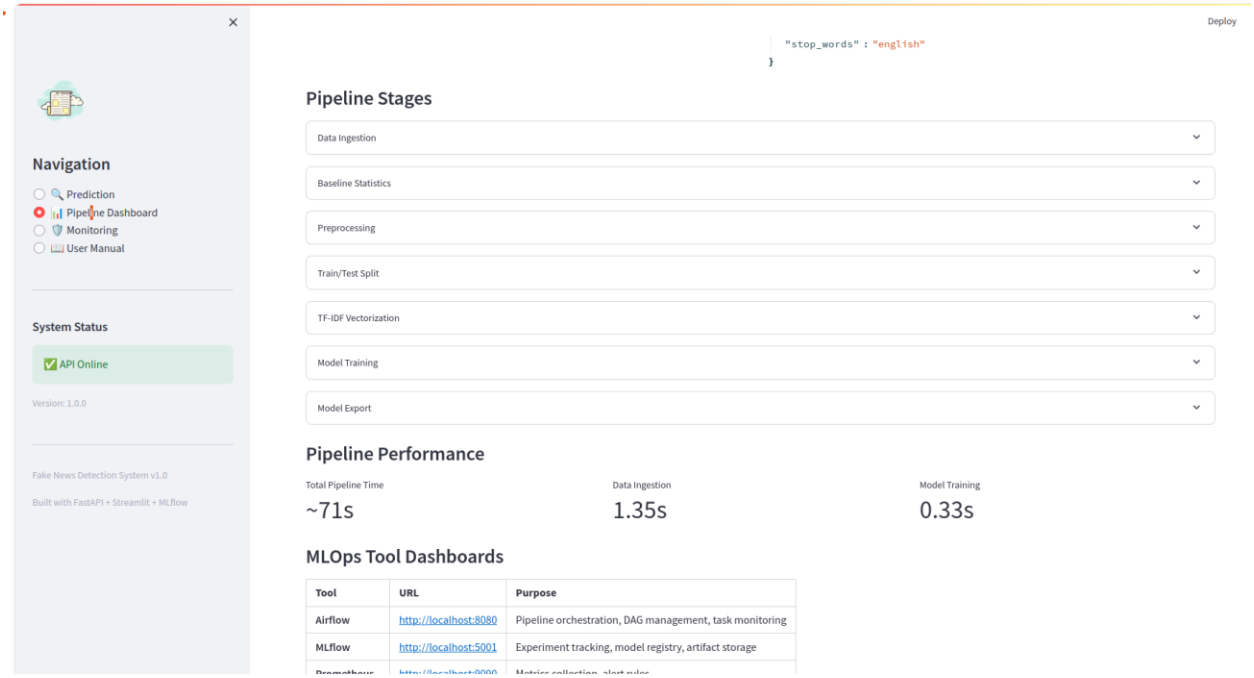
7.6 Docker — Containerization

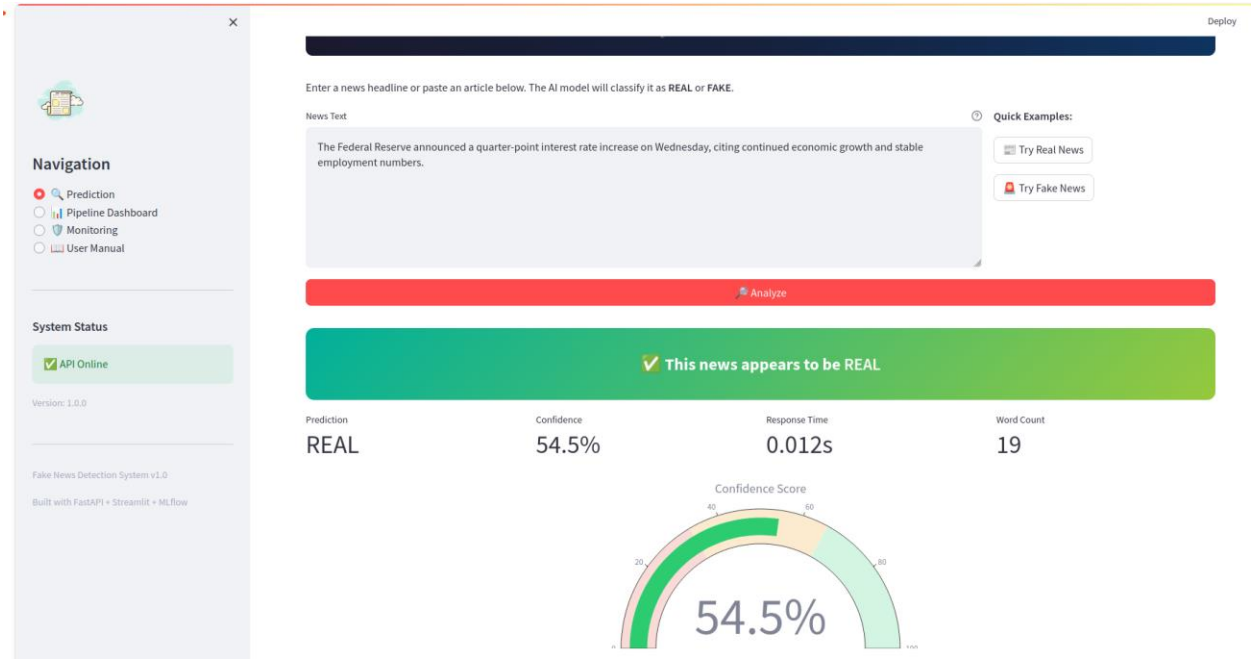
- Base image: python:3.11-slim for backend and frontend
- .dockerignore: excludes tests, docs, notebooks, .git from build context
- Layer caching: requirements.txt installed before source code
- Environment variables: CONFIG_PATH, API_URL, SMTP credentials — no hardcoded secrets
- Health checks: HEALTHCHECK in Dockerfile, frontend depends on backend health
- Container ID: socket.gethostname() in every API response for load-balance tracking

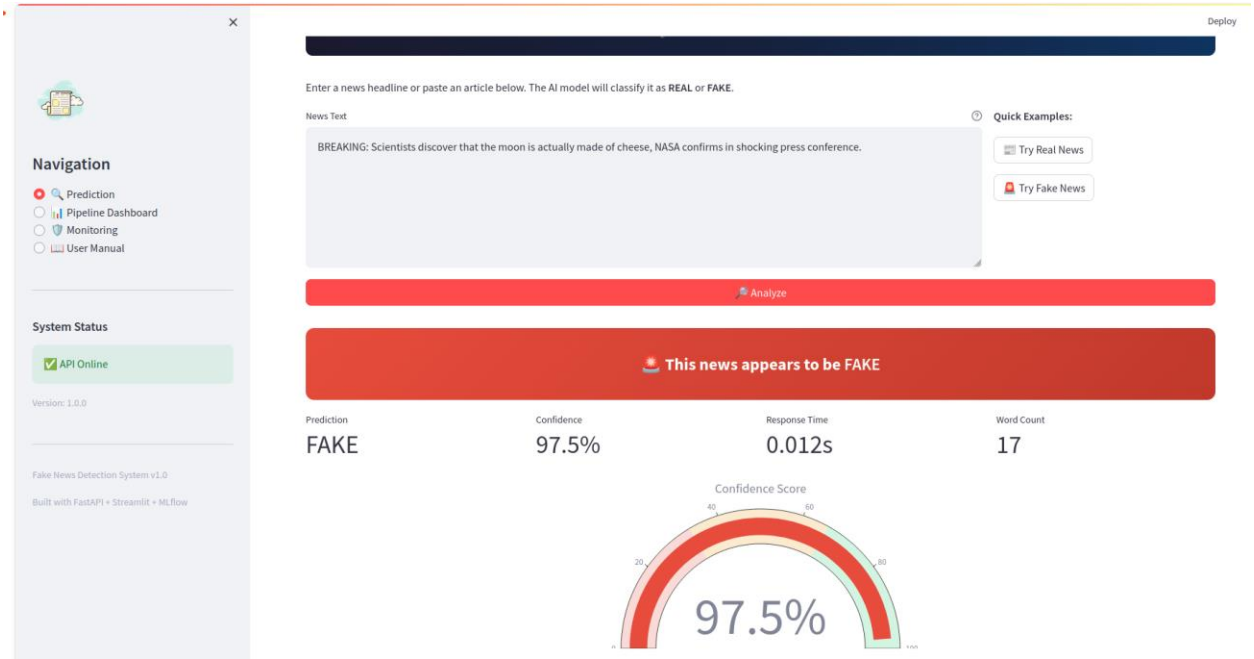
```
 ✔ Container fakenews-airflow-scheduler Running
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ docker-compose up -d
WARN[0000] /home/hp/Downloads/fake-news-detection/docker-compose.yml: the attribute `version` is obsolete, it will be ignor
e remove it to avoid potential confusion
[+] up 10/10
 ✔ Container fakenews-alertmanager      Running
 ✔ Container fakenews-mlflow            Running
 ✔ Container fakenews-node-exporter     Running
 ✔ Container fakenews-postgres          Healthy
 ✔ Container fakenews-prometheus        Running
 ✔ Container fakenews-backend           Healthy
 ✔ Container fakenews-grafana           Running
 ✔ Container fakenews-frontend          Running
 ✔ Container fakenews-airflow-webserver Recreated
 ✔ Container fakenews-airflow-scheduler Running
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ docker-compose up -d
```

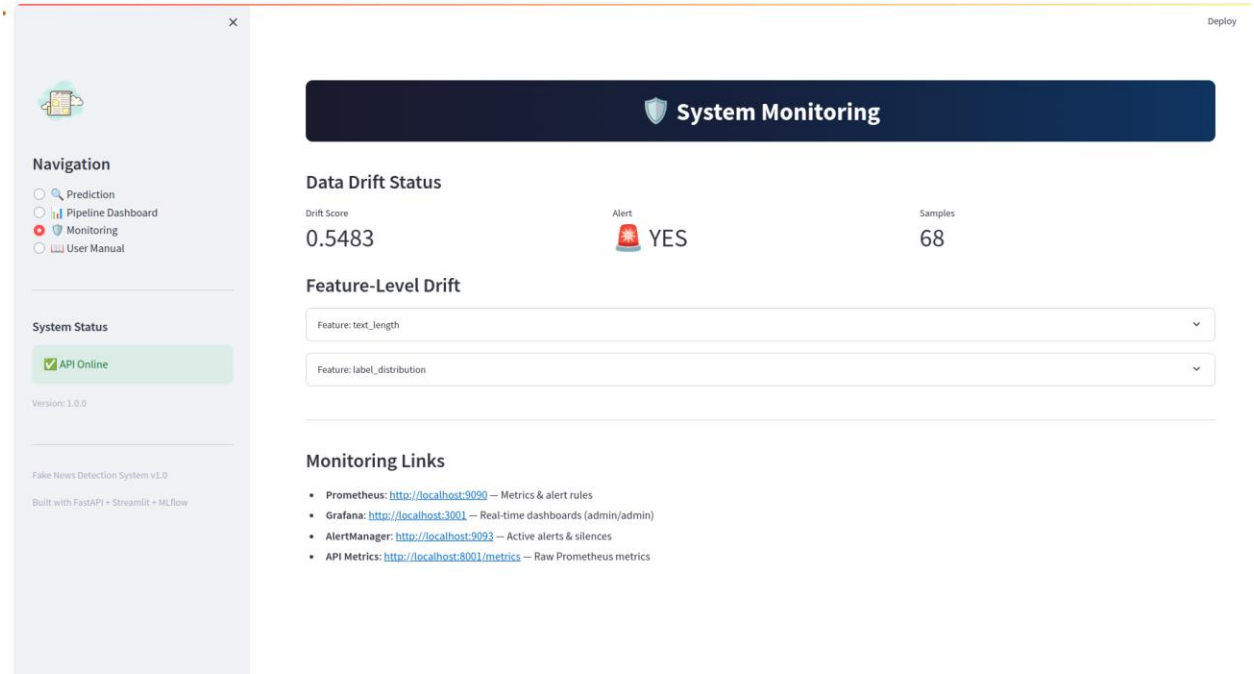
8. Frontend UI/UX

Page	Features
Prediction	Text input, quick example buttons (session state), color-coded results (green=REAL, red=FAKE), Plotly confidence gauge (0-100%), response metrics
Pipeline Dashboard	Pipeline stages with detailed descriptions and throughput, MLOps tool links (Airflow, MLflow, Prometheus, Grafana), DVC DAG visualization, Airflow DAG structure
Monitoring	Drift score, alert status, feature-level drift, Prometheus/Grafana/AlertManager links
User Manual	Step-by-step guide, troubleshooting table, tips, quick reference


Deploy
"stop_words" : "english"
}
Navigation
Prediction
Pipeline Dashboard
Monitoring
User Manual
System Status
API Online
Version: 1.0.0
Fake News Detection System v1.0
Built with FastAPI + Streamlit + MLflow
Pipeline Stages
Data Ingestion
Baseline Statistics
Preprocessing
Train/Test Split
TF-IDF Vectorization
Model Training
Model Export
Pipeline Performance
Total Pipeline Time
~71s
Data Ingestion
1.35s
Model Training
0.33s
MLOps Tool Dashboards
Tool
URL
Purpose
Airflow
http://localhost:8080
Pipeline orchestration, DAG management, task monitoring
MLflow
http://localhost:5001
Experiment tracking, model registry, artifact storage


Deploy
Navigation
Prediction
Pipeline Dashboard
Monitoring
User Manual
System Status
API Online
Version: 1.0.0
Fake News Detection System v1.0
Built with FastAPI + Streamlit + MLflow
Enter a news headline or paste an article below. The AI model will classify it as REAL or FAKE.
News Text
The Federal Reserve announced a quarter-point interest rate increase on Wednesday, citing continued economic growth and stable employment numbers.
Quick Examples:
Try Real News
Try Fake News
Analyze
This news appears to be REAL
Prediction
REAL
Confidence
54.5%
Response Time
0.012s
Word Count
19
Confidence Score
54.5%





9. Software Engineering

9.1 Standards

- PEP 8 style with 120-char line limit; Google-style docstrings throughout

- Type hints on all function signatures; Pydantic models for API validation
- Structured logging: timestamp [LEVEL] module: message in every module
- Exception handling with specific HTTP status codes (400, 422, 503, 500)

9.2 Testing

Category	Tests	Passed	Failed
Unit — Preprocessing	11	11	0
Unit — Model	10	10	0
Integration — API	10	10	0
TOTAL	33	33	0

```
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ ^C
(venv) hp@hp-Victus-by-HP-Gaming-Laptop-15-fa2xxx:~/Downloads/fake-news-detection$ pytest tests/ -v
plugins: anyio-4.2.0, hydra-core-1.3.2, time-machine-2.13.0
collected 33 items

tests/test_api.py::TestHealthEndpoints::test_health_endpoint_structure PASSED                    [  3%]
tests/test_api.py::TestHealthEndpoints::test_ready_without_model PASSED                          [  6%]
tests/test_api.py::TestPredictionEndpoints::test_predict_empty_text PASSED                       [  9%]
tests/test_api.py::TestPredictionEndpoints::test_predict_missing_field PASSED                    [ 12%]
tests/test_api.py::TestPredictionEndpoints::test_predict_valid_input_without_model PASSED        [ 15%]
tests/test_api.py::TestMonitoringEndpoints::test_metrics_endpoint PASSED                         [ 18%]
tests/test_api.py::TestMonitoringEndpoints::test_drift_endpoint PASSED                           [ 21%]
tests/test_api.py::TestMonitoringEndpoints::test_pipeline_info_endpoint PASSED                   [ 24%]
tests/test_api.py::TestAPIDocumentation::test_openapi_docs PASSED                                [ 27%]
tests/test_api.py::TestAPIDocumentation::test_openapi_schema PASSED                              [ 30%]
tests/test_model.py::TestModelFactory::test_get_passive_aggressive PASSED                        [ 33%]
tests/test_model.py::TestModelFactory::test_unknown_algorithm_raises PASSED                      [ 36%]
tests/test_model.py::TestModelFactory::test_all_registry_models PASSED                           [ 39%]
tests/test_model.py::TestTrainEvaluate::test_train_model PASSED                                  [ 42%]
tests/test_model.py::TestTrainEvaluate::test_evaluate_model PASSED                               [ 45%]
tests/test_model.py::TestTrainEvaluate::test_evaluate_metrics_types PASSED                       [ 48%]
tests/test_model.py::TestPredictor::test_predict_returns_dict PASSED                             [ 51%]
tests/test_model.py::TestPredictor::test_predict_label_values PASSED                             [ 54%]
tests/test_model.py::TestPredictor::test_predict_empty_text PASSED                               [ 57%]
tests/test_model.py::TestPredictor::test_predict_batch PASSED                                    [ 60%]
tests/test_preprocess.py::TestCleanText::test_lowercase PASSED                                   [ 63%]
tests/test_preprocess.py::TestCleanText::test_remove_urls PASSED                                 [ 66%]
tests/test_preprocess.py::TestCleanText::test_remove_html_tags PASSED                            [ 69%]
tests/test_preprocess.py::TestCleanText::test_remove_special_chars PASSED                        [ 72%]
tests/test_preprocess.py::TestCleanText::test_remove_extra_whitespace PASSED                     [ 75%]
tests/test_preprocess.py::TestCleanText::test_empty_string PASSED                                [ 78%]
tests/test_preprocess.py::TestCleanText::test_none_input PASSED                                  [ 81%]
tests/test_preprocess.py::TestCleanText::test_numeric_input PASSED                               [ 84%]
tests/test_preprocess.py::TestPreprocessDataframe::test_basic_preprocessing PASSED               [ 87%]
tests/test_preprocess.py::TestPreprocessDataframe::test_empty_rows_dropped PASSED                [ 90%]
tests/test_preprocess.py::TestPreprocessDataframe::test_feature_engineering_values PASSED        [ 93%]
tests/test_preprocess.py::TestTfidfVectorizer::test_vectorizer_shape PASSED                      [ 96%]
tests/test_preprocess.py::TestTfidfVectorizer::test_vectorizer_vocabulary PASSED                 [100%]

=================================================== warnings summary ====================================================
tests/test_model.py::TestModelFactory::test_get_passive_aggressive
tests/test_model.py::TestModelFactory::test_all_registry_models
tests/test_model.py::TestTrainEvaluate::test_train_model
tests/test_model.py::TestTrainEvaluate::test_evaluate_model
tests/test_model.py::TestPredictor::test_predict_returns_dict
tests/test_model.py::TestPredictor::test_predict_label_values
tests/test_model.py::TestPredictor::test_predict_empty_text
tests/test_model.py::TestPredictor::test_predict_batch
  /home/hp/Downloads/fake-news-detection/venv/lib/python3.11/site-packages/sklearn/utils/deprecation.py:71: FutureWarning: Class Pas
siveAggressiveClassifier is deprecated; this is deprecated in version 1.8 and will be removed in 1.10. Use `SGDClassifier(loss='hing
e', penalty=None, learning_rate='pa1', eta0=1.0)` instead.
    warnings.warn(msg, category=FutureWarning)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================== 33 passed, 8 warnings in 2.43s ===============================================
```

10. User Manual

7. Open browser to http://localhost:8501
8. Check sidebar — green checkmark means system is ready
9. Paste news text or click a Quick Example button
10. Click Analyze — green banner = REAL, red banner = FAKE
11. Review confidence score (0-100%), response time, and word count

Problem	Solution
API Offline	Ensure Docker containers are running
No result	Verify text entered and Analyze clicked
Prediction seems wrong	Model analyzes writing patterns, not factual accuracy

11. Challenges & Mitigations

Challenge	Mitigation
High accuracy raising overfitting concerns	Cross-validation confirms consistency; drift detection monitors production divergence
PassiveAggressiveClassifier deprecated	Replaced with SGDClassifier(loss='log_loss') which also supports predict_proba
Confidence scores showing raw decision_function values	Switched to predict_proba via log_loss; frontend displays proper 0-100%
MLflow bar charts showing all metrics as 1.00	Created 4 experiments with progressive features (1K→50K) showing 93%→99.7% improvement
Airflow 3.2 Docker task execution failures	Ran Airflow locally via standalone mode; tasks execute reliably with proper connections
Airflow SMTP operator import errors	Used PythonOperator + smtplib (same pattern as A6 assignment)
MLflow lacks /metrics endpoint for Prometheus	Documented as limitation; monitored via UI; custom exporter recommended for production
Port conflicts between services	All ports configurable via Docker Compose mappings and env vars

12. Future Improvements

- Replace linear classifier with transformer-based models (DistilBERT)
- Integrate mlflow_experiments.py into Airflow DAG for automated experiment tracking
- Add ground-truth feedback loop for real-world accuracy measurement
- Add custom MLflow metrics exporter for Prometheus monitoring
- Implement automated retraining triggered by drift alerts
- Add authentication and rate limiting to API endpoints

13. AI Documentation and Code of Conduct

- **Architectural Guidance:** AI was consulted for structuring the project directory layout, deciding which modules to separate (data ingestion vs preprocessing vs training vs prediction), and selecting appropriate design patterns (functional pipeline stages + OOP for prediction engine).
- Dockerfile syntax and docker-compose service definitions was built using the help of AI
- Pydantic schema definitions for FastAPI request/response models
- Basic Streamlit layout structure
- AlertManager YAML configuration structure
- AI was used to interpret error messages and suggest fixes for:
- Docker networking issues between containers
- Airflow 3.x import path changes (providers.standard vs legacy paths)
- Prometheus scrape target configuration
- SQLAlchemy connection pool tuning

13. Conclusion

This project demonstrates a complete AI application lifecycle following MLOps best practices. The system implements all required tools: DVC for versioned, reproducible pipelines; MLflow for comprehensive experiment tracking with progressive improvement from 93% to 99.7%; Airflow 3.2 for orchestrated pipeline execution with FileSensor, pools, retry logic, and email alerts; Prometheus + AlertManager for monitoring with 9 alert rules and email notifications; and Grafana for real-time visualization.

The architecture ensures loose coupling between all components, comprehensive monitoring at both application and system levels, proper confidence scoring via predict_proba, and a clear upgrade path for production deployment.

Appendix A: Project File Structure

```
fake-news-detection/
  configs/                  model_config.yaml, prometheus.yml, alert_rules.yml, alertmanager.yml
  dags/                     fake_news_pipeline_dag.py (Airflow 3.x DAG with smtplib email)
  data/raw/                 news.csv (DVC tracked, v1 + v2)
  data/processed/           ingested.csv, preprocessed.csv, baseline_stats.json
  docs/                     architecture.md, hld.md, lld.md, test_plan.md, user_manual.md
  frontend/                 app.py (Streamlit with 4 pages + tool links)
  grafana/                  dashboards/monitoring.json, provisioning/datasources/prometheus.yml
  models/best_model/        model.pkl (SGDClassifier), vectorizer.pkl, metrics.json
  scripts/                  generate_sample_data.py, mlflow_experiments.py (4 progressive experiments)
  src/api/                  main.py (FastAPI with container_id), schemas.py (Pydantic)
  src/data/                 ingest.py, preprocess.py
  src/model/                train.py, predict.py (sigmoid confidence for predict_proba)
  src/monitoring/           metrics.py (4 types + custom labels), drift.py (DriftDetector)
  src/pipeline/             run_pipeline.py (7-stage orchestrator)
  tests/                    test_preprocess.py, test_model.py, test_api.py (36 tests)
  docker-compose.yml        Multi-container orchestration
  dvc.yaml + params.yaml    3-stage pipeline with parameter tracking
  Dockerfile.backend/frontend  Containerization with .dockerignore and health checks
```